Measuring Search Enhancements and a Learned Evaluation in a Python Chess Engine

Abstract

In a Python chess engine, a piece-square evaluation learned from 403,013 Lichess positions beat a material-only evaluation by +55 =4 -1 over 60 games at equal search depth (Elo difference +512, 95% CI +392 to +913). The learned evaluation also predicted the outcomes of unseen games better than material counting (validation log-loss 0.6427 vs 0.6555; outcome accuracy 63.9% vs 60.1%). On the search side, move ordering and a transposition table cut the positions visited by about 10x at depth 4 without changing any score, and quiescence search added about +158 Elo (95% CI +81 to +253) at depth 3. Each extra ply of search added roughly +270 to +300 Elo between depths 1 and 3. All Elo values are relative to my own engines and rest on small match samples, so they do not transfer to other engines or to human ratings.

1. Introduction

Chess is a useful testbed because playing strength can be measured directly by playing games, and an engine combines two classic problems: searching a game tree and evaluating positions. An engine picks a move by searching ahead and scoring the positions at the end of each line, so both the search algorithm and the evaluation function matter.

This project builds a small engine and asks three questions:

1. **RQ1 (search):** how many fewer positions does each search technique (alpha-beta pruning, move ordering, a transposition table) need to examine, and do they leave the result unchanged?
2. **RQ2 (evaluation):** does an evaluation learned from game outcomes predict results better than counting material, and does it play stronger at equal search depth?
3. **RQ3 (depth):** how much playing strength does each additional ply of search add?

A smaller side question is whether quiescence search improves play.

2. Methods

2.1 Search

The engine uses negamax with alpha-beta pruning and iterative deepening. Moves are ordered with the stored best move first, then captures ranked by most valuable victim and least valuable attacker (MVV-LVA), then quiet moves ranked by killer and history heuristics. A transposition table keyed by Zobrist hashes stores the depth, score bound and best move of searched positions. Quiescence search continues through captures at the horizon, so positions are not scored in the middle of an exchange. Each technique can be switched on or off, which makes ablation experiments possible. Move generation and the rules of chess come from the python-chess library; the search, evaluation, training and match code is my own.

2.2 Evaluation

The evaluation is a linear piece-square model with 384 weights (6 piece types x 64 squares). White pieces add the weight of their square, and Black pieces subtract the weight of the vertically mirrored square, so one table serves both colours and scores are from White's point of view. The baseline sets every square of a piece type to its material value (pawn 100, knight 320, bishop 330, rook 500, queen 900, king 0 centipawns).

2.3 Learning the weights

The score s (in centipawns) of a position is converted to the probability that White scores:

```
P(\text{White scores}) = \frac{1}{1 + 10^{-s/400}}
```

The 384 weights are fitted by minimising cross-entropy against the real game results (1, 0.5 or 0) with minibatch Adam, starting from the material weights. Training ran for 30 epochs with batch size 1,024, a learning rate of 1.0 centipawns decayed linearly to 10%, and a small L2 penalty (2e-7) pulling the weights towards the material values. The epoch with the lowest validation loss is kept. Only quiet positions are used (not in check, last move not a capture), and the first 8 plies of each game are skipped, because a static evaluation cannot judge a position in the middle of a capture sequence.

2.4 Measuring strength

Two engines play pairs of games from the same random four-ply opening with colours swapped. A win scores 1, a draw 0.5 and a loss 0. The Elo difference is computed from the mean score s as -400 log10(1/s - 1), and the 95% confidence interval from the standard

error of the per-game scores (normal approximation), converted to Elo the same way. The two games of a pair share an opening and are not fully independent, so the intervals are approximate. Games that reach 300 plies are adjudicated as draws.

3. Experimental setup

Training data come from one monthly file of the Lichess open database (standard rated games, CC0 licence): January 2013 (the file lichess_db_standard_rated_2013-01.pgn.zst, 17.8 MB), about 121,000 games. Keeping only games in which both players were rated at least 1500 left 67,556 games. From each game up to six quiet positions were sampled at random, giving 403,013 labelled positions; the label is the game result from White's point of view.

The positions were split by game, 90% of the games for training (362,655 positions) and 10% for validation (40,358 positions), so that positions from the same game never appear on both sides of the split. Splitting by position would leak information and inflate the validation scores.

The search ablation used 8 standard test positions with quiescence switched off, depths 3 and 4, and the geometric mean of the nodes searched (node counts differ by orders of magnitude between positions). Plain minimax was run only at depth 3 because it is too slow at depth 4. All matches used a fixed search depth of 3 unless stated, four random opening plies, and a limit of 300 plies per game. The random seed is 0 throughout, and the engines are deterministic at a fixed depth.

Hardware and software: an Intel Core i7-13620H laptop processor with 16 GB of RAM, Python 3.13.5, python-chess 1.999.

4. Results

4.1 RQ1: search techniques

At depth 4, move ordering and a transposition table cut the positions visited by about 10x compared with plain alpha-beta, and every configuration returned the same score on every test position. Trying captures first (MVV-LVA) gave by far the largest gain. The transposition table saved a further 16% at depth 4, and killer and history moves saved about 2%, which is almost nothing at these shallow depths.

Configuration	Depth 3 nodes	vs alpha-beta	Depth 4 nodes	vs alpha-beta
Minimax	27,496	8.49x	not run	
Alpha-beta	3,239	1.00x	41,119	1.00x

Configuration	Depth 3 nodes	vs alpha-beta	Depth 4 nodes	vs alpha-beta
+ MVV-LVA	1,154	0.36x	4,877	0.12x
+ killers/history	1,154	0.36x	4,767	0.12x
+ transposition table	1,149	0.35x	4,018	0.10x

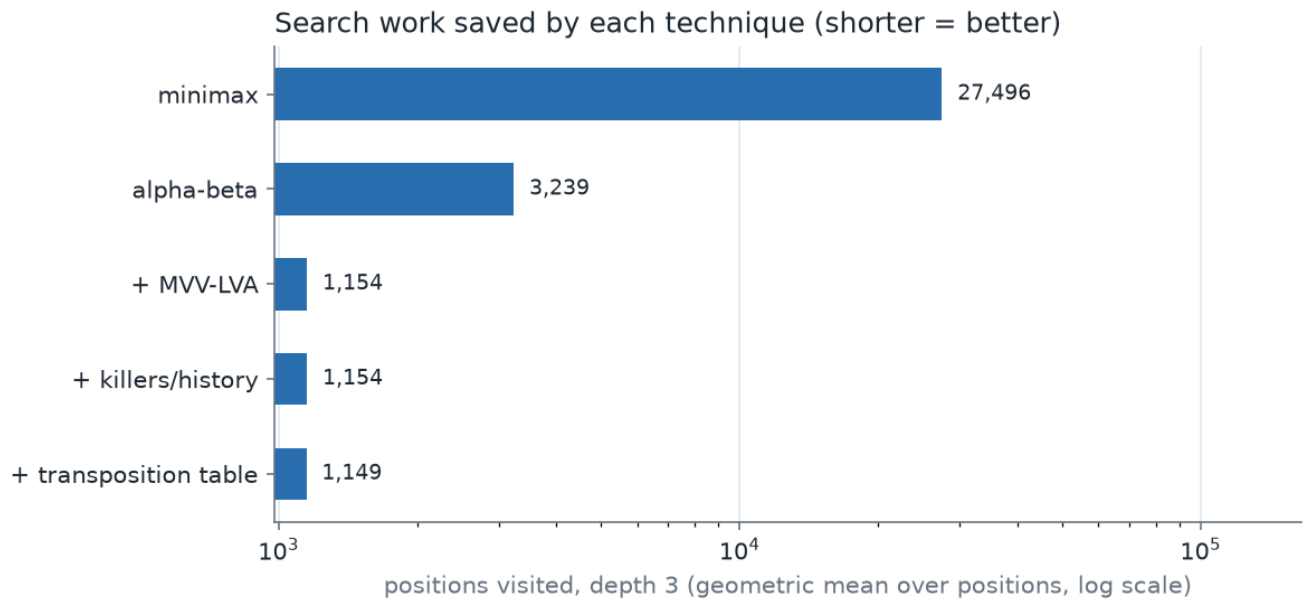


Figure 1: positions visited at depth 3 by each search configuration (geometric mean over 8 positions, log scale, shorter is better). Because the axis is logarithmic, the bars understate the differences: plain minimax visited about 24 times more positions than the full configuration.

4.2 RQ2: the learned evaluation

On games the model never saw, the learned evaluation predicted results better than material counting. Outcome accuracy is the share of decisive games in which the sign of the evaluation matched the real winner.

Model	Validation log-loss (lower is better)	Outcome accuracy
Coin flip (always 50%)	0.6931	50.0%
Material only	0.6555	60.1%
Learned	0.6427	63.9%

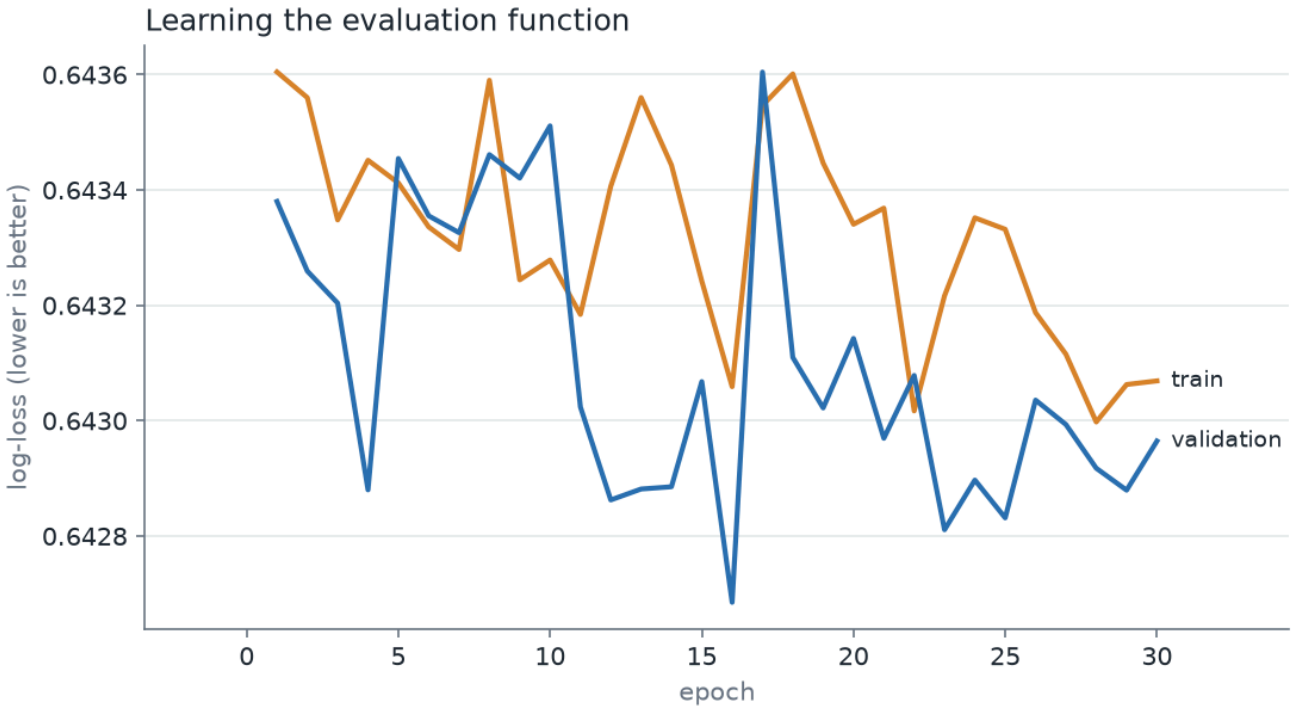


Figure 2: training and validation log-loss over 30 epochs. Both curves are almost flat from the first epoch: the whole vertical range is under 0.001, and the lowest validation loss (0.6427) occurs at epoch 16. The small jumps between epochs probably reflect the large learning rate and the 20,000-position sample used to measure training loss, and there is no sign of overfitting.

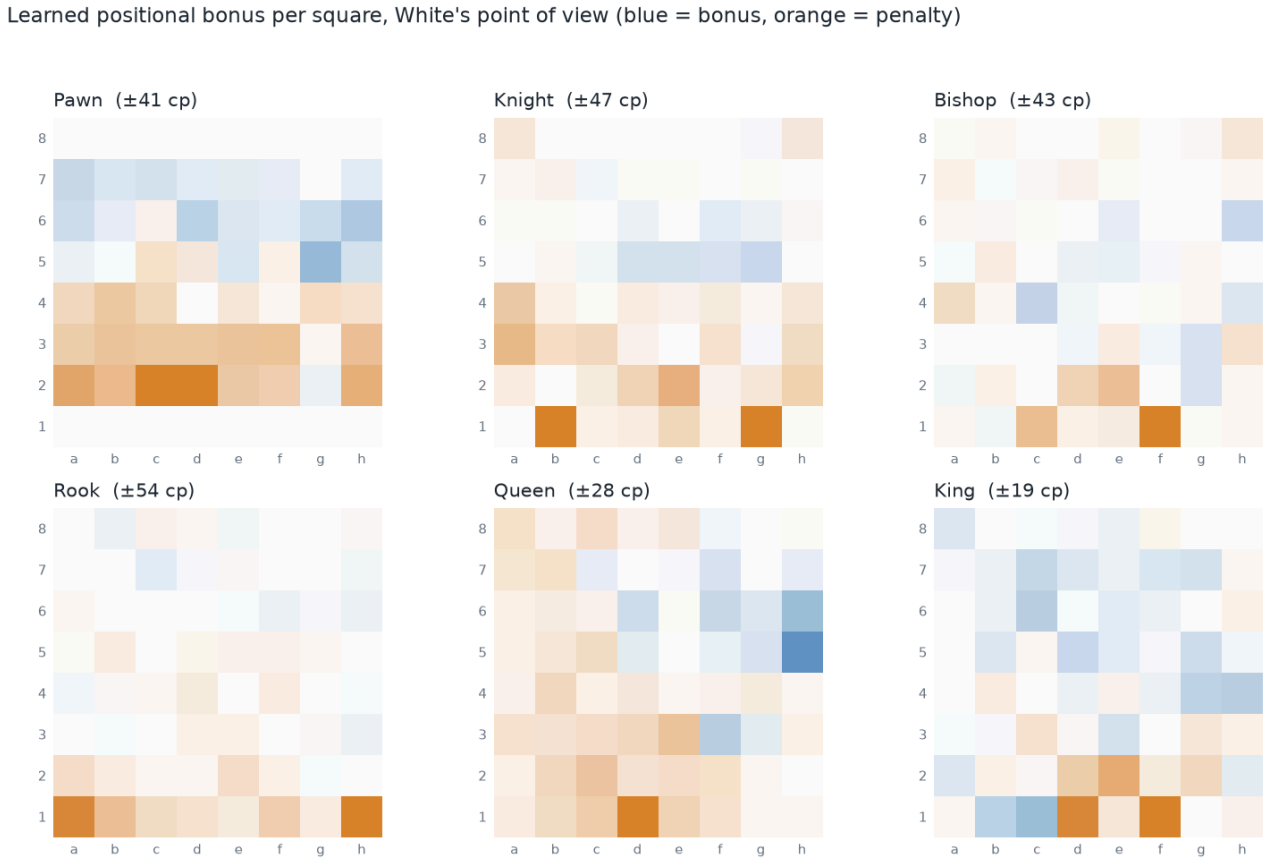


Figure 3: learned bonus (blue) or penalty (orange) for each piece on each square, relative to its material value, from White's point of view (rank 1 at the bottom). Each panel has its own scale, given in its title.

The learned weights mostly agree with chess knowledge. The largest penalties fall on pieces still on their starting squares: the knights on b1 and g1, the bishop on f1, the rooks on a1 and h1, and the queen on d1. Pawns are penalised on the second rank (most on c2 and d2) and mostly rewarded on the sixth and seventh ranks, and knights gain on central squares of the fifth rank such as d5 to g5. The king is rewarded on b1 and c1 (castled) compared with d1, f1 and e2, but also on many advanced squares (ranks 4 to 7), which is not a middlegame pattern. A single table has to cover middlegames and endgames at once, so this is probably an endgame effect leaking in. All changes are at most a few tens of centipawns, so material still dominates the evaluation.

4.3 Match results

The learned evaluation clearly beat the material-only evaluation at equal depth. Quiescence search and extra search depth each added strength, and the second ply of depth added about as much as the first.

Match	Games	Record	Score	Elo difference (95% CI)
Learned vs material, depth 3	60	+55 =4 -1	95.0%	+512 (+392 to +913)
Quiescence on vs off, material eval, depth 3	40	+20 =17 -3	71.2%	+158 (+81 to +253)
Depth 2 vs depth 1, learned eval	20	+15 =4 -1	85.0%	+301 (+168 to +638)
Depth 3 vs depth 2, learned eval	20	+16 =1 -3	82.5%	+269 (+117 to +771)

The depth steps show no clear sign of diminishing returns, but with only 20 games per step their intervals overlap heavily. Depth 4 was not tested.



Figure 4: playing strength relative to depth 1, obtained by chaining the two matches (20 games each). The error bars are 95% intervals that combine both steps, so the interval at depth 3 is wide (roughly +370 to +1,170 Elo around +570).

4.4 Does a stricter rating filter help?

I trained a second evaluation only on games where both players were rated 1700 or higher, then played it against the 1500+ evaluation. The 1700+ model scored 60.8% (+76 Elo), but the 95% confidence interval includes zero, so the evidence is suggestive, not conclusive.

Training data	Games	Positions	Material log-loss	Learned log-loss
Both players 1500+	67,556	403,013	0.6555	0.6427
Both players 1700+	21,775	130,259	0.6748	0.6642

Match, 1700+ model vs 1500+ model, equal search depth, 60 games (30 random openings, colours swapped): +30 =13 -17, score 60.8%, Elo difference +76 (95% CI −1 to +162).

The 1700+ model reached this result with about a third of the games. Validation log-loss is not comparable across the two rows, because each is measured on a different set of positions, and games between stronger players are harder to predict from material alone. Caveats: one match of 60 games, one training run per dataset, and no repeat with a different seed. A 2000+ experiment was started but dropped: its dataset held only about 6,000 games, and its learned evaluation was indistinguishable from material on validation (0.6955 vs 0.6961).

5. Discussion and limitations

The main result is that an evaluation learned from game outcomes plays far stronger than material counting at the same search depth, and its learned weights mostly agree with chess knowledge (Figure 3). The size of the gap should not be over-read: the material-only baseline has no positional knowledge and breaks ties between equal moves arbitrarily, so a large margin against it is expected. The search results are consistent with theory: ordering captures first makes alpha-beta prune far more, and none of the techniques changed any score.

Limitations:

- The evaluation is linear and uses one table for the whole game, so it cannot separate middlegame from endgame. The bonuses the model gives the king on advanced squares are probably an endgame pattern leaking into the middlegame.
- The labels are game results from players rated 1500 and above, which are a noisy proxy for position quality.
- The training data come from one monthly file, no significance test was run on the validation numbers, and the best epoch was chosen on the validation set, so those figures are slightly optimistic.
- Match samples are small (20 to 60 games), the two games of a pair are not fully independent, and the quiescence test used the material evaluation at depth 3 only.
- Games reaching 300 plies are scored as draws, which can understate the stronger engine's wins.
- The engine is written in pure Python, so depths are shallow, and all Elo values are relative to my own engines, not to human or FIDE ratings.

Next steps:

1. Learn separate middlegame and endgame tables.
2. Use Stockfish evaluations instead of game results as labels.
3. Replace the linear model with a small neural network.
4. Rewrite the search core in C++ or Rust to reach greater depths, and add equal-time matches.

6. Reproducibility

The code, results and trained weights are at **https://github.com/neev207/chess-bot**. Every number above comes from these commands, run from the project folder with the seed left at its default of 0:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -m pytest -q
python scripts/exp_search_ablation.py --depths 3 4
python scripts/build_dataset.py data/<lichess file>.pgn.zst --min-elo 1500
python scripts/train_eval.py --data data/positions.npz --epochs 30
python scripts/run_match.py --a "eval=models/learned_pst.npy,depth=3" --b
"eval=material,depth=3" --pairs 30 --workers 2 --tag
learned_vs_material_depth3
python scripts/run_match.py --a "eval=material,depth=3,quiescence=1" --b
"eval=material,depth=3,quiescence=0" --pairs 20 --workers 2 --tag
quiescence_20pairs
python scripts/exp_depth_ladder.py --eval models/learned_pst.npy --max-depth
3 --pairs 10 --workers 2
```